- **this——默认值**
  - 普通函数
    - 普通函数的调用方式决定了 `this` 的值，即【谁调用 `this` 的值指向谁】，如下代码所示：

```
<script>
  // 普通函数
  function sayHi() {
    console.log(this);
  }
  // 函数表达式
  let sayHello = function () {
    console.log(this);
  }

  // 函数的调用方式决定了 this 的值
  sayHi(); // window
  window.sayHi(); // window

    // 普通对象
  let user = {
    name: '小明',
    walk: function () {
      console.log(this);
    }
  };
  // 动态为 user 添加方法
  user.sayHi = sayHi;
  user.sayHello = sayHello;

  // 函数调用方式，决定了 this 的值
  user.sayHi();
  user.sayHello();
</script>
```

    - 注： 普通函数没有明确调用者时 `this` 值为 `window`，严格模式下没有调用者时 `this` 的值为 `undefined`。
  - 箭头函数
    - 箭头函数中的 `this` 与普通函数完全不同，也不受调用方式的影响，事实上箭头函数中并不存在 `this`！箭头函数中访问的 `this` 不过是箭头函数所在作用域的 `this` 变量。

```
<script>
  console.log(this); // 此处为 window
  // 箭头函数
  let sayHi = function() {
    console.log(this); // 该箭头函数中的 this 为函数声明环境中 this 一致
  }

  // 普通对象
  let user = {
    name: '小明',
    // 该箭头函数中的 this 为函数声明环境中 this 一致
    walk: () => {
      console.log(this);
    },

    sleep: function () {
      let str = 'hello';
      console.log(this);
      let fn = () => {
        console.log(str);
        console.log(this); // 该箭头函数中的 this 与 sleep 中的 this 一致
      }
      // 调用箭头函数
      fn();
    }
  }

  // 动态添加方法
  user.sayHi = sayHi;

  // 函数调用
  user.sayHi();
  user.sleep();
  user.walk();
</script>
```

- 在开发中【使用箭头函数前需要考虑函数中 `this` 的值】，事件回调函数使用箭头函数时，`this` 为全局的 `window`，因此DOM事件回调函数不推荐使用箭头函数，如下代码所示：

```
<script>
  // DOM 节点
  let btn = document.querySelector('.btn');

  // 箭头函数 此时 this 指向了 window
  btn.addEventListener('click', () => {
    console.log(this);
  })

  // 普通函数 此时 this 指向了 DOM 对象
  btn.addEventListener('click', function () {
    console.log(this);
  })
</script>
```

- 同样由于箭头函数 `this` 的原因，基于原型的面向对象也不推荐采用箭头函数，如下代码所示：

```
<script>
  function Person() {

  }

  // 原型对像上添加了箭头函数
  Person.prototype.walk = () => {
    console.log('人都要走路...');
    console.log(this); // widow
  }

  let p1 = new Person();
  p1.walk();
</script>
```

- class类
  - 构造函数、实例属性、实例方法中引用【实例】

```
class Animal {
  constructor(name){
```

```
    this.name = name
  }
  instanceProp = this
  instanceFn(){
    return this
  }
}

const animal = new Animal('Orange')
console.log(animal === animal.instanceProp) // true
console.log(animal === animal.instanceFn()) // true
```

- 静态方法、静态属性中引用【类】

```
class Animal {
  static staticProp = new this()
  static staticFn(){
    return this
  }
}

console.log(Animal.staticProp) // 实例
console.log(Animal === Animal.staticFn()) // true
```

- 总结

```
class Animal {
  constructor(){
    // this引用【实例】
  }
  static staticProp // this引用【类】
  static staticFn(){
    // this引用【类】
  }
  instanceProp // this引用【实例】
  instanceFn(){
    // this引用【实例】
  }
```

```
}
```

- **this——定义值**
  - 以上归纳了普通函数和箭头函数中关于 `this` 默认值的情形，不仅如此 JavaScript 中还允许指定函数中 `this` 的指向，有 3 个方法可以动态指定普通函数中 `this` 的指向：
  - call
    - 使用 `call` 方法调用函数，同时指定函数中 `this` 的值，使用方法如下代码所示：

```
<script>
  // 普通函数
  function sayHi() {
    console.log(this);
  }

  let user = {
    name: '小明',
    age: 18
  }

  let student = {
    name: '小红',
    age: 16
  }

  // 调用函数并指定 this 的值
  sayHi.call(user); // this 值为 user
  sayHi.call(student); // this 值为 student

  // 求和函数
  function counter(x, y) {
    return x + y;
  }

  // 调用 counter 函数，并传入参数
  let result = counter.call(null, 5, 10);
  console.log(result);
</script>
```

        - 总结：
            - `call` 方法能够在调用函数的同时指定 `this` 的值
            - 使用 `call` 方法调用函数时，第1个参数为 `this` 指定的值
            - `call` 方法的其余参数会依次自动传入函数做为函数的参数
    - apply
        - 使用 `call` 方法调用函数，同时指定函数中 `this` 的值，使用方法如下代码所示：

```
<script>
  // 普通函数
  function sayHi() {
    console.log(this);
  }

  let user = {
    name: '小明',
    age: 18
  }

  let student = {
    name: '小红',
    age: 16
  }

  // 调用函数并指定 this 的值
  sayHi.apply(user); // this 值为 user
  sayHi.apply(student); // this 值为 student

  // 求和函数
  function counter(x, y) {
    return x + y;
  }

  // 调用 counter 函数，并传入参数
  let result = counter.apply(null, [5, 10]);
  console.log(result);
</script>
```

        - 总结：
            - `apply` 方法能够在调用函数的同时指定 `this` 的值
            - 使用 `apply` 方法调用函数时，第1个参数为 `this` 指定的值

        - `apply` 方法第2个参数为数组，数组的单元值依次自动传入函数做为函数的参数
    - bind
        - `bind` 方法并不会调用函数，而是创建一个指定了 `this` 值的新函数，使用方法如下代码所示：

```
<script>
  // 普通函数
  function sayHi() {
    console.log(this);
  }

  let user = {
    name: '小明',
    age: 18
  }

  // 调用 bind 指定 this 的值
  let sayHello = sayHi.bind(user);

  // 调用使用 bind 创建的新函数
  sayHello();
</script>
```

        - 注：`bind` 方法创建新的函数，与原函数的唯一的变化是改变了 `this` 的值。
    - 改变this三个方法总结：

```
call: fun.call(this, arg1, arg2,......)
apply: fun.apply(this, [arg1, arg2,......])
bind: fun.bind(this, arg1, arg2,......)
```

        - 相同点：
            - 都可以用来改变this指向，第一个参数都是this指向的对象
        - 不同点：
            - call和apply：都会使函数执行，但是参数不同
            - bind：不会使函数执行，参数同call

- **浅拷贝**
    - 含义：只拷贝最外面层的拷贝方式

```
 let obj = {
    uname: '张三丰',
    age: 22,
    sex: '男',
    color: ['red', 'blue', 'yellow', 'pink'],
    message: {
        index: 1,
        score: 99
    }
}
let newObj = {};
Object.assign(newObj, obj);
console.log(obj, newObj);
```

- **深拷贝**
  - 含义：所有层都拷贝的方式

```
let obj = {
    uname: '张三丰',
    age: 22,
    sex: '男',
    color: ['red', 'blue', 'yellow', 'pink'],
    message: {
        index: 1,
        score: 99
    }
}
let newObj = {};

function kaobei(newObj, obj) {
    for (let key in obj) {
        if (obj[key] instanceof Array) { // obj[key] 是数组
            // obj[key]是数组，遍历
            newObj[key] = [];
            kaobei(newObj[key], obj[key]);
        } else if (obj[key] instanceof Object) { // obj[key]是对象
            // obj[key]是对象，遍历
            newObj[key] = {};
```

```
            kaobei(newObj[key], obj[key]);
        } else {
            newObj[key] = obj[key];
        }
    }
}
kaobei(newObj, obj);
obj.message.score = 123;
console.log(obj, newObj);
```